Phase 2: Extended Literature Review and Research Methodology Development

GNN's for Topology-Aware AIOps: Network Anomaly Detection and Root-Cause Localization

1. Research focus and continuity with Phase 1

Phase 1 established that network key performance indicators (KPIs) should not be treated as unrelated time series. Congestion and faults spread along communication links, so a useful AIOps model needs information about both the network graph and how performance changes over time. Our initial direction was a topology aware graph neural network (GNN) for anomaly detection and root-cause localization, with failure prediction as a possible extension.

The original review considered topology-aware self-supervised KPI monitoring by Luo et al., graph-based alarm correlation and diagnosis by Zhou et al., and graph construction for network anomaly detection by Cressant et al. [P1–P3] Phase 2 retains that direction rather than proposing a different topic. It asks a narrower, testable question:

Can a spatio-temporal GNN trained with limited fault labels identify anomalous network behaviour, warn about developing performance failures, and localize the responsible node or link when the underlying topology changes?

The project will study three related tasks, but not treat them as interchangeable: (i) detecting an anomaly in the current observation window; (ii) forecasting degradation before a measurable failure threshold is crossed; and (iii) ranking likely faulty nodes or links after an incident occurs. Abrupt, unheralded link cuts will be evaluated for detection and localization, not misleadingly described as predictable failures.

2. Extended literature review: ten additional studies

The ten studies below are additional to the three used in Phase 1. They were selected because they report a clear method and evaluation, and together cover network failures, spatio-temporal modelling, graph construction, limited topology visibility, and fault diagnosis. Papers on microservices or general dynamic graphs are used as methodological comparisons, not presented as telecom field trials. The results quoted are the *authors' findings*, not results obtained in this project. Full titles and persistent identifiers appear in the references.

Studies from 2025: Hasan et al.'s GenTrap [1] combines learnable spatial aggregation with a time-series transformer to predict radio-link failures. Chen et al. [2] show how prototype-based graph learning can make optical network fault decisions more interpretable. Unyi et al. [3] use service graphs and simulated probabilistic failures to forecast system-level risk. Yuan et al.'s NetEventCause [4] provides an important counterexample to topology-dependent RCA: it infers root alarms from event timing when network structure is unavailable. Zoubir and Missaoui [5] examine graph input representation for intrusion detection, while Abdaljalil et al. [6] model anomalies in temporally evolving graphs.

Studies from 2026: Papry et al. [7] combine a GNN and transformer for anomaly detection and root-cause analysis in a simulated 5G radio access network. Sun et al. [8] explicitly diagnose node and edge faults together. Moens et al. [9] show why changing service topology matters for context-aware AIOps. Zou et al. [10] combine multiple observability signals for incident diagnosis, highlighting both the advantages and the runtime cost of richer reasoning.

Selection and interpretation note. Journal quartiles vary by ranking system, subject category, and assessment year; the department should confirm whether it uses Scimago/Scopus or JCR. All ten entries below are published research articles from identified journals. We avoid assuming that a claim about one network type transfers automatically to another.

Cross-paper comparison (2025–2026)

Methods and reported outcomes are paraphrased; a dash or "not stated" means the consulted published summary did not provide a defensible number.

#	PAPER - VENUE	YEAR	DATASET - SOURCE	PRE-PROCESSING/ GRAPH DESIGN	METHOD	EVALUATION	REPORTED FINDING	LIMITATION RELEVANT TO OUR PROJEC
1	GenTrap: GNN-Transformer radio-link failure prediction; IEEE TMLC	2025	Real rural and urban 5G-related deployments; 2.6 million radio KPI observations	Spatial association of weather stations with radio sites; historical radio and weather windows	GNN weather aggregation plus transformer sequence encoder	Precision, recall, F1; generalization	F1 of 0.93 (rural), 0.79 (urban); improvement over LSTM-based comparison	Predicts radio-link failure; does not jointly evaluate general node/link RCA on changing wired topologies.
2	Interpretable optical-network fault detection and localization; JOCN	2025	Optical-network performance monitoring and fault scenarios	Network and monitoring readings represented as graph; node/device prototypes	Multi-task graph prototype learning (MT-GPL), Monte Carlo tree search prototypes	Fault prediction/localization accuracy and explanation inspection	More than 6.5% higher prediction accuracy than an MLP comparator	Optical-specific impairment classes and physical semantics; transfer to generic IP faults untested.
3	Explainable GNN fault fore- casting in cloud service debug- ging; IEEE TNSM	2025	Simulated service meshes: trees, DAGs and feature- rich structures	Failure probabilities sampled for graph nodes; simulation labels from Markov decision processes	Message-passing GNN with explainer for influential components	MAE, R2; comparison with MLP and regression	Reported full-node system- risk R2 ≈ 0.996 in an explanation analysis	Mainly simulation-derived fault behaviour; forecasts system risk rather than network-interface RCA.
4	NetEventCause: topology-free RCA of network alarms; IEEE TNNLS	2025	Synthetic alarm sequences and operational airport- platform data (>200,000 entities)	Historical alarms converted to timestamped event sequences; no supplied topology	Multivariate neural temporal point process plus attribu- tion	Root-alarm localiza- tion and event-chain recovery; likelihood quality	Outperformed comparison RCA and event-model methods in reported tests	Recovers alarm-event causes; lacks explicit physical-link semantics or prospective KPI failure forecasting.
5	GNNs with scattering trans- form for network anomaly detection; EAAI	2025	Network-intrusion datasets; contaminated / uncontaminated settings	Flow-to-edge graph; wavelet scattering features; DeepWalk node initialization; 10% subsampling	Scattering Transform Edge Graph (STEG), edge-centric GraphSAGE	Intrusion/anomaly comparison across benchmark methods	Improved network anomaly detection relative to tested alternatives	Attack-flow classification is not identical to operational node/link fault localization; subsampling may

								change graph statistics.
6	Deep temporal and structural embeddings in dynamic graphs; IEEE OJCS	2025	Weibo, Reddit, Enron, DGraph	Construct temporal graph snapshots and node-feature sequences	GConvGRU + LSTM + reconstruction autoencoder (T-StructGAD)	ROC-AUC, 12 competing methods	ROC-AUC 0.857 (Weibo), 0.575 (Reddit), 0.759 (Enron), 0.634 (DGraph)	Strong methodological analogy, but benchmarks are general interaction graphs, not routed network telemetry.
7	GNN–Transformer anomaly detector and RCA in RANs (Simba); IEEE TNSM	2026	Calibrated, 3GPP-aligned Simu5G scenarios; concur- rent faults and different RAN loads	Simulated KPI windows, UE/base-station relations and fault labels	Spatial GNN + temporal transformer; joint detection and RCA	F1 under multiple fault and load set-tings	F1 at least 0.95 across evaluated settings; re-ported 18% improvement over a detection baseline	RAN simulator realism is domain-bound; generalization to unseen network families and scarce labels remains to be measured.
8	Hybrid node–edge fault diagnosis under HPMC∗; IEEE Transactions on Networking	2026	Multiple interconnection topologies and real-network tests	Convert diagnostic syndromes to edge-enhanced graph features	Hybrid-GNN: GraphSAGE for simultaneous node/edge diagnosis	Node accuracy, link accuracy/precisi on	>99% node and >97% link accuracy within stated diagnostic bound	Requires syndrome-style diagnostic observations; not evaluated as KPI-based temporal warning system.
9	Context-aware anomaly detection with dynamic knowledge graphs; IEEE TNSM	2026	Open-source microservice benchmark with monitoring and injected faults	Evolving dependency/hosting graph from resource and network observations	Context-aware dynamic-knowledge-graph anomaly detection	Detection com-parisons to non-contextual baselines	Consistently exceeded non-contextual comparisons in reported benchmark	Microservice topology, not direct router/link RCA; evidence for topology shift is helpful but task differs.
10	Graph-augmented multi-agent RCA in AIOps; Computers, Materials & Continua	2026	Train Ticket, Online Boutique, MSDS, AIOps Challenge 2025, OpenTelemetry Demo	Align logs and traces; build anomaly-fusion graph	Graph-constrained, multi-agent semantic diagnosis	Precision, recall, F1; runtime	Mean RCA F1 88.4%; 4.6 percentage points above stated strongest comparison; mean runtime 21.4 s/incident	Extra modalities and reasoning add dependency and latency; not a lightweight GNN-only network monitoring method.

3. Cross-paper synthesis and refined gap

The ten papers support graph-based network analysis, but they solve different parts of the operational problem. GenTrap [1] shows that relational context can improve *forecasting*; Simba [7] and Hybrid-GNN [8] show that graphs help *localization*; Moens et al. [9] show why *topology changes* must be represented. NetEventCause [4] also cautions against assuming the topology is always visible. The strongest opening is therefore not a claim that GNN-based AIOps has never been studied. It is the lack of a controlled, common evaluation of *joint detection, warning and localization under topology shift and restricted labels*.

Three gaps supported by three focal studies

Potential gap	GenTrap [1]	Simba [7]	Dynamic KG [9]	Research opportunity
Unseen-topology transfer	Tests generalized site/weather relations, not different wired network families.	Examines several RAN fault/load scenarios in Simu5G.	Captures evolving microservice dependencies, not physical switch/link topology.	Hold out whole topology families; measure performance after link/node additions and removals.
End-to-end diagnosis	Forecasts radio-link failure risk, not root-cause ranking.	Detects and diagnoses faults but does not establish lead-time forecasting as the same task.	Detects anomalies with context; forecasting plus component RCA is not its central objective.	Separate outputs and metrics for present anomaly, future degradation and incident origin.
Dependence on labels and domain	Uses known historical radio failures for supervised learning.	Calibrated synthetic faults yield controlled labels but may not span unseen failure behaviour.	Injected benchmark faults support evaluation, not proof of low-label operation in network emulation.	Pretrain on normal telemetry, then vary the fraction of labelled faults; compare with fully supervised learning.

Refined research gap. Recent work shows that GNNs can exploit topology and time-dependent telemetry for network performance analysis. Yet it remains insufficiently established how a single compact model performs across these three AIOps tasks when (a) a network changes structure, (b) the entire test topology is unseen, and (c) few diagnosed faults are available during training. This matters because operators cannot assume that new deployments will reproduce training conditions, nor that every rare failure has already been labelled. Our proposed study will test these conditions explicitly rather than relying on high accuracy obtained from random splits of a single emulated topology.

4. Research objectives and hypotheses

Objectives:

1. Detect anomalous behavior from a time-window of network KPIs and connectivity.
2. Forecast *developing performance degradation* at a stated horizon.
3. Rank the fault-origin node or link, including when symptoms appear elsewhere.
4. Quantify the effect of topology shift, missing labels and model complexity.

Hypotheses:

1. A topology-aware temporal model will improve incident-level F1 or PR-AUC against a temporal-only baseline on held-out topologies.
2. Using physical links together with learned KPI dependencies will increase root-cause Top-3 accuracy relative to either relation type alone.
3. Pretraining on healthy data will reduce the amount of labeled fault data needed for stable diagnosis. All three are hypotheses, *not reported outcomes*.

5. Proposed research methodology

Step 1. Define the research input

Represent each observation time t as a network graph $Gt = (Vt, Et)$ and associated measurements $X_t \in \mathbb{R}^{|V_t| \times d}$. A node is a router/switch; an edge is an observed physical or logical communication link. Edge features include utilization, capacity, loss and measured path or link delay when available. The temporal input is $\{(Gt - L + 1, Xt - L + 1), \ldots, (Gt, Xt)\}$, with a pilot window of $L = 12$ observations at 5-second intervals. Use a node/link failure-origin label only where injection logs give an unambiguous location.

Step 2. Data collection

Create a reproducible emulation using **Mininet + Open vSwitch**, Python traffic control, iperf3 background flows and active probes. Build star, ring, tree and mesh networks at least two sizes, varying host traffic and link capacities. Record counters and interface state at 5-second intervals and keep configuration manifests. Inject congestion, bandwidth reduction, increasing delay, packet loss and link disconnection; for appropriate tests include a switch/process outage. Run normal traffic episodes under comparable loads. Fault time, affected component, severity and episode ID are recorded by the injection script. The dataset is *proposed to be collected*; no experiments or sample counts are claimed here.

Step 3. Data preprocessing

Synchronize snapshots to the collection interval, convert cumulative counters to nonnegative rates after handling resets, and discard corrupt probes rather than inventing plausible measurements. Fit scaling parameters on training episodes only. Encode unavailable measurements with a mask so missing telemetry is distinguishable from zero traffic. Segment by episode and preserve causal timestamps; never let a sliding window overlap training and test partitions. Mark the first few seconds after a fault separately from steady-state behaviour so detection delays can be computed.

Step 4. Feature engineering and graph representation

For each device, collect traffic rate, interface errors/drops, queue information when supported, link status and aggregates from attached interfaces. For each link, retain capacity and traffic/loss indicators; add rolling change and deviation-from-baseline features derived strictly from past samples. Maintain *two graph views*: (i) the known physical adjacency $A^{\text{phys}}{}_t$ and (ii) a sparse learned relation graph $A^{\text{rel}}{}_t$ from training-normal KPI dependencies. Relations inferred from future or test data are prohibited. Compare combined, physical-only and learned-only graphs; cap learned edges to reduce spurious relationships.

Step 5. Proposed model / method

Begin with a modest shared spatio-temporal encoder: a two-layer GraphSAGE or graph-attention block processes neighbours in each graph snapshot, followed by a GRU over time. Three outputs share the learned representation: $\hat{a}_t$ (anomaly probability), $\hat{p}_{t+h}$ (probability that a measurable KPI degradation crosses a predeclared threshold within horizon h) and $rt(v)/rt(e)$ (node/link scores for RCA). Compare purely supervised training with optional masked-KPI reconstruction pretraining on normal traffic. A weighted, masked objective is

$$L = \lambda_a L_{detect} + \lambda_f L_{forecast} + \lambda_r L_{RCA} + \lambda_s L_{selfsup}$$

Only apply a task's loss to episodes with the corresponding valid labels. If multi-task learning hurts a task, report that result and compare separate heads rather than presenting joint training as automatically superior.

Step 6. Baseline methods

Compare against (i) fixed KPI thresholds, (ii) Isolation Forest using the same aggregated features, (iii) an LSTM/GRU with no topology, (iv) a static GCN/GraphSAGE with no temporal recurrence, and (v) a physical graph spatio-temporal GNN without learned relations or pretraining. Use equivalent splits, feature availability, tuning budget and alert thresholds across the models. Include an attribution baseline that ranks components by their local standardized KPI deviation.

Step 7. Experimental design

Construct *episode-disjoint* train/validation/test sets and run two protocols. **Within-topology:** later episodes from known topology instances, with chronological separation. **Unseen-topology:** leave one complete topology family out for testing, rotating the held-out family across folds. Training/validation may not see its snapshots, identifiers, or labels. Within each protocol, vary fault type/severity, traffic pattern, network size and available labelled fraction (100%, 25%, 10%). Keep a fixed alert threshold selected only on validation episodes. Repeat stochastic training with at least five random seeds if resources allow; log hardware and runtime.

Step 8. Evaluation metrics

Detection: macro-F1, precision/recall, PR-AUC, false alarms per hour, and detection delay from injection to first sustained alert. Forecasting: PR-AUC, precision/recall at a prespecified horizon (initially h = 30 or 60 seconds), calibration/Brier score, and useful lead time before threshold crossing. RCA: Top-1/Top-3 hit rate and mean reciprocal rank for a labelled originating node or link. Operational cost: inference latency, memory and model size. Report results by scenario rather than pooling hard and easy faults into a single flattering number.

Step 9. Statistical and comparative analysis

Summarize per-episode scores, with mean and 95% bootstrap intervals resampled *by episode*, not by overlapping time window. Pair competing methods on exactly the same held-out episodes and report paired differences and confidence intervals. Apply a paired permutation test or Wilcoxon signed-rank test to episode-level differences when suitable; avoid a significance claim from repeated windows of one fault. Also provide performance by topology family and by fault severity.

Step 10. Ablation and component analysis

Remove, one at a time: temporal recurrence, physical adjacency, learned KPI relations, normal-only pretraining and RCA head. Vary the graph neighbourhood depth, sampling interval, window length and label budget. Report whether topology additions/removals or missing telemetry trigger unstable outputs. The ablations test which components earn their complexity and which are unnecessary.

Step 11. Error and failure analysis

Inspect false alarms caused by legitimate traffic surges, missed weak faults, faults producing symptoms at distant nodes, mislabelled root locations and noisy counters. Separate “anomaly detected but wrong origin” from “not detected”. Audit leakage risks such as node IDs, a fixed injection schedule, or a graph built using future statistics. Discuss the failure cases with concrete topology/telemetry plots after experiments, rather than guessing their frequency in advance.

Step 12. Final research output

Deliver: versioned Mininet configurations and collection scripts; raw and processed graph snapshots; a fault/episode manifest; baseline and proposed-model code; held-out topology evaluation with uncertainty; the ablation and error-analysis tables; and a short operator-facing example that reports *what changed, when, and which node or link is most suspect*. These are target outputs for the research stage, not completed Phase 2 experiments.

6. Validity, feasibility and scope

Feasibility. A Mininet testbed avoids dependence on a restricted operator dataset and provides unambiguous injected-fault labels. Its limits should remain visible: software emulation cannot reproduce all telecom radio behavior, hardware faults, and live operational noise. Use a small two-layer GNN first; add learned relations and self-supervised training only if baselines justify them. The absence of realistic pre-failure indicators for an instantaneous hard cut is a physical limit, not a modelling flaw that should be hidden.

Threats to validity. Synthetic traffic can be overly repetitive; unbalanced faults can inflate accuracy; graph node identifiers can leak scenarios; and fault injections can create detectable timing artifacts. Use varied background loads, episode-level partitioning, normal runs without injection, and held-out network families to reduce these risks. Even with those controls, conclusions must be limited to the emulated network conditions tested.

7. Research methodology architecture

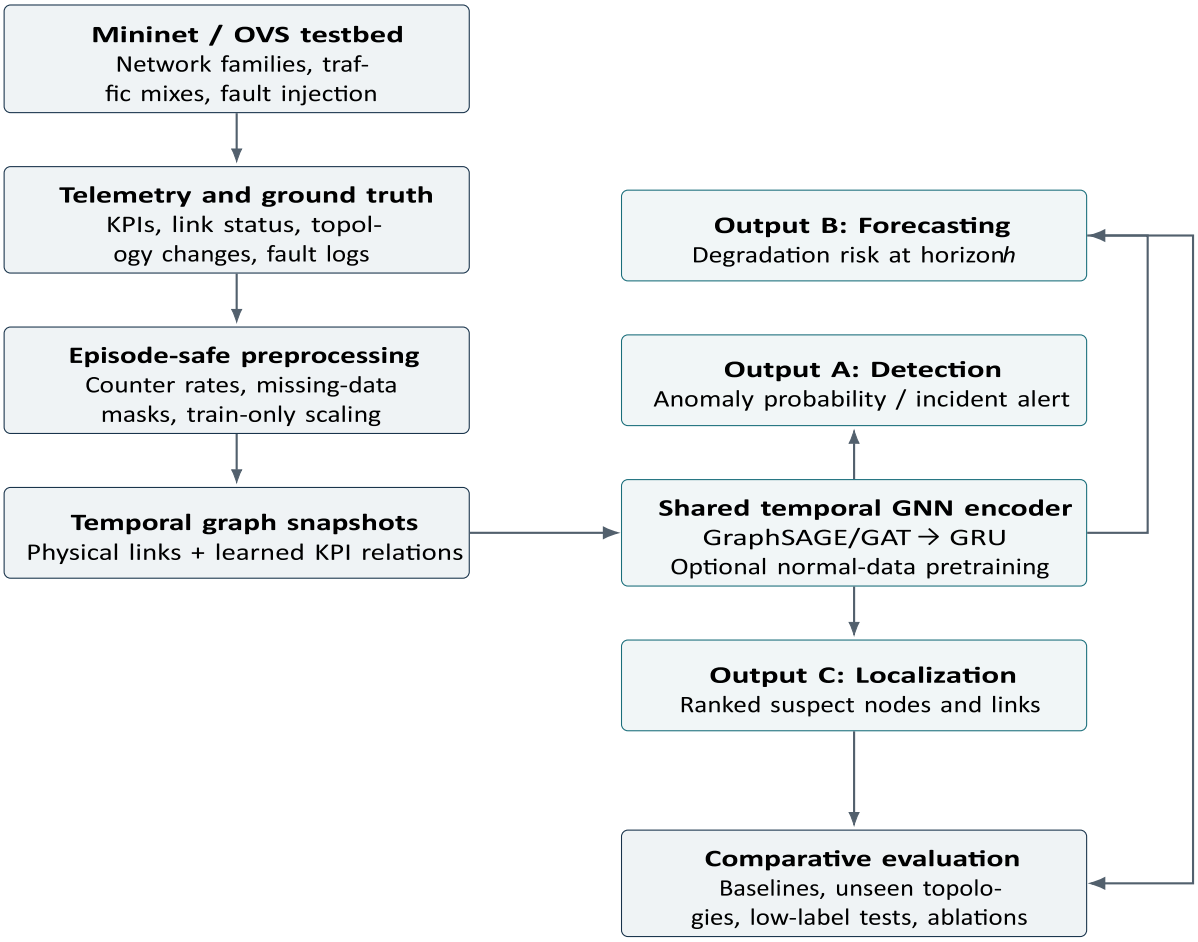


Figure 1. Proposed architecture. Training and evaluation are separate: only training/validation data determine model parameters, derived correlations, feature scales and alert thresholds. RCA uses incident evidence; forecasting uses only observations available before the prediction horizon.

8. Intended Contribution

The main contribution would be an evaluation framework and compact model for graph-based network operations that tests difficult but practical constraints: changing topology, limited fault labels and the distinction between predicting a developing failure and diagnosing one that has already happened. The study would count as useful even if a simpler GRU or a fixed-topology baseline matches the proposed model, because the resulting comparison would show when topology-aware learning actually pays for its complexity. No detection, forecasting or localization scores are claimed until the experiments are run.

References

[1] K. Hasan, K. Papry, T. Trappenberg, and I. Haque, "A Generalized GNN-Transformer-Based Radio Link Failure Prediction Framework in 5G RAN," *IEEE Transactions on Machine Learning in Communications and Networking*, vol. 3, pp. 710–724, 2025. doi:10.1109/TMLCN.2025.3575368.

[2] X. Chen, X. Chen, and Z. Zhu, "Interpretable optical network fault detection and localization with multi-task graph prototype learning," *Journal of Optical Communications and Networking*, vol. 17, no. 9, pp. D73–D82, 2025. doi:10.1364/JOCN.562633.

[3] D. Unyi, E. Rigó, B. Gyires-Tóth, and R. Lovas, "Explainable GNN-Based Approach to Fault Forecasting in Cloud Service Debugging," *IEEE Transactions on Network and Service Management*, vol. 22, no. 6, pp. 5640–5657, 2025. doi:10.1109/TNSM.2025.3602223.

[4] Z. Yuan et al., "NetEventCause: Event-Driven Root Cause Analysis for Large Network System Without Topology," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 10, pp. 18983–18994, 2025. doi:10.1109/TNNLS.2025.3574316.

[5] A. Zoubir and B. Missaoui, "Graph Neural Networks with scattering transform for network anomaly detection," *Engineering Applications of Artificial Intelligence*, vol. 150, article 110546, 2025. doi:10.1016/j.engappai.2025.110546.

[6] S. Abdaljalil et al., "Deep Temporal and Structural Embeddings for Robust Unsupervised Anomaly Detection in Dynamic Graphs," *IEEE Open Journal of the Computer Society*, vol. 6, pp. 1100–1109, 2025. doi:10.1109/OJCS.2025.3584942.

[7] K. Papry, C. Boeira, K. Hasan, Y. Ju, Z. Zhu, and I. Haque, "A Graph Neural Network and Transformer-Based Anomaly Detector and Root Cause Analyzer in Radio Access Networks," *IEEE Transactions on Network and Service Management*, vol. 23, pp. 7191–7202, 2026. doi:10.1109/TNSM.2026.3728303.

[8] X. Sun et al., "A Graph Neural Network Approach for Hybrid Node-Edge Fault Diagnosis in Interconnection Networks Under the HPMC* Model," *IEEE Transactions on Networking*, vol. 34, pp. 3918–3931, 2026. doi:10.1109/TON.2026.3670807.

[9] P. Moens, B. Steenwinckel, F. Ongenae, B. Volckaert, and S. Van Hoecke, "Toward Context-Aware Anomaly Detection for AIOps in Microservices Using Dynamic Knowledge Graphs," *IEEE Transactions on Network and Service Management*, vol. 23, pp. 1970–1988, 2026. doi:10.1109/TNSM.2026.3652304.

[10] H. Zou, Y. Zhao, X. Chen, L. Wang, J. Yu, and L. Yuan, "Graph-Augmented Multi-Agent Robust Root Cause Analysis in AIOps," *Computers, Materials & Continua*, vol. 88, no. 1, article 40, 2026. doi:10.32604/cmc.2026.077908.

Phase 1 background – not counted toward the ten additional papers

[P1] D. Luo et al., "Topology-aware self-supervised multivariate KPI anomaly detection and root cause node localization for network operations," *Scientific Reports*, 2026. doi:10.1038/s41598-026-69392-8.

[P2] B. Zhou et al., "Root cause analysis of cloud platform faults based on anomaly correlation graph and graph neural network," *Engineering Applications of Artificial Intelligence*, vol. 174, article 114451, 2026. doi:10.1016/j.engappai.2026.114451.

[P3] K. Cressant, F. Larroca, S. Secci, and P. B. Velloso, "On Graph Design for GNN-Based Network Anomaly Detection," *IEEE Transactions on Network and Service Management*, vol. 23, pp. 4122–4134, 2026. doi:10.1109/TNSM.2026.3684653.